

# A comparative study of finite-element techniques for analyzing complex geometries

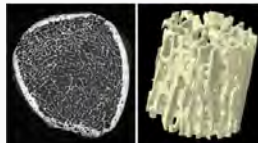
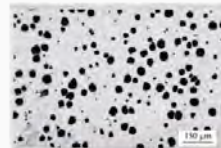
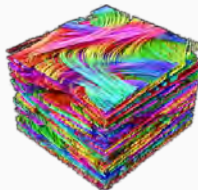
---

Nagham Mawassy   **Grégory Legrain**   Valentine Rey   Xiaodong Liu

Nantes Université, École Centrale de Nantes, CNRS, GeM, UMR 6183, F-44000 Nantes, France

# Motivation

- Image-based computations:
  - Material science
  - Biomedical engineering
  - Retro-engineering
- Various acquisition methods: CT, MRI, 3D scanning, etc.
- Geometry–CAE integration is a persistent bottleneck
  - Meshing  $\approx$  up to 80% of analysis time
  - Gaps, intersections, overlaps degrade quality



## Objective

Compare three more or less non-standard approaches (*needing minimal geometrical pre-treatment*), all built on a **level-set** description of the geometry:

**Voxel-based FEM**

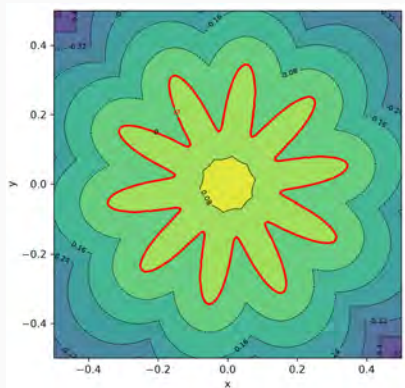
**CDFEM**

**Fictitious domain**

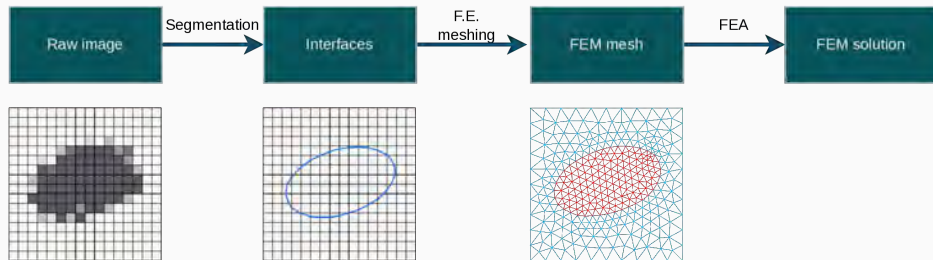
- Same code base (mfem) for a "fair" comparison (and for the challenge ;-))
- 2D & 3D, scalar (thermal) & vector (elasticity) problems
- Increasing geometric complexity
- Any input geometry convertible to a level-set (image, mesh, CAD)

# Implicit geometry: the level-set method

- Interface  $\partial\Omega$  = iso-zero of a scalar function  $\psi(x)$  (signed distance function here)
- Naturally handles topological changes (merging, splitting)
- Natural regularization of the geometry (no gaps, overlaps, intersections)
- Obtained from segmentation, or recomputed from a stl mesh
- Interpolated on a background mesh
- Usually linear interpolation, but higher-order possible



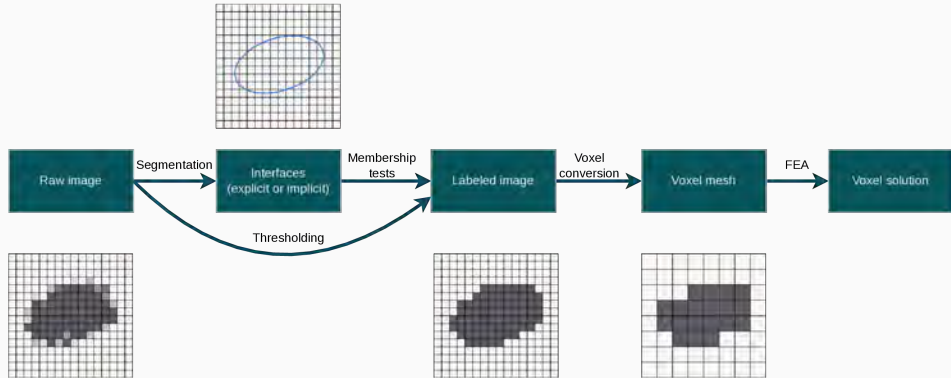
# Finite elements



- + Robust, well-established (low-order), flexible, parallelizable, ecosystem
- Geometrical robustness (meshing), efficiency for moving interfaces

# Voxel finite elements

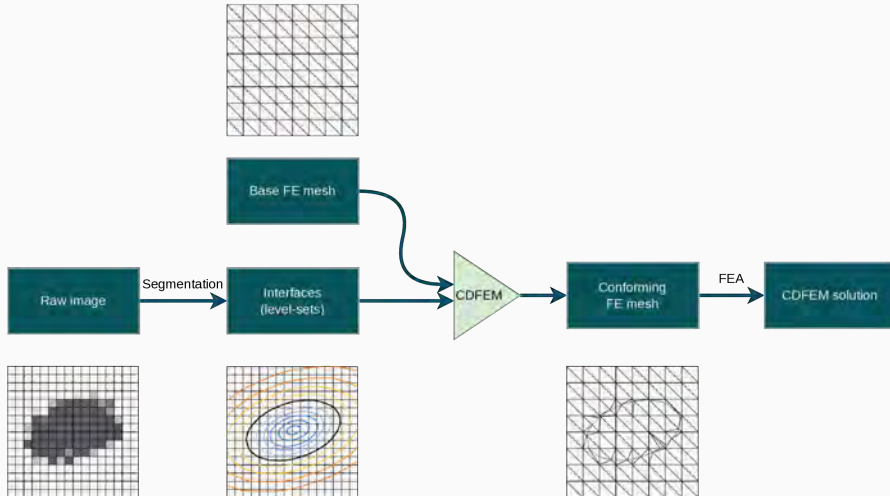
[Hollister & Kikuchi 1994] [Guldberg et al. 1998] [Boyd & Muller 2006] [Liu et al. 2022]



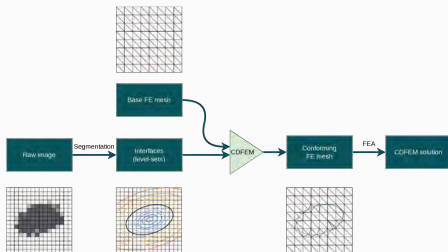
- + Very robust (geometry, numerics), matrix-free, parallelizable
- Geometrical accuracy, stress oscillations, sub-optimal convergence

# Continuous decomposition FEM (cdfem)

[Noble et al. 2010] [Kramer & Noble 2014] [Fries 2017]

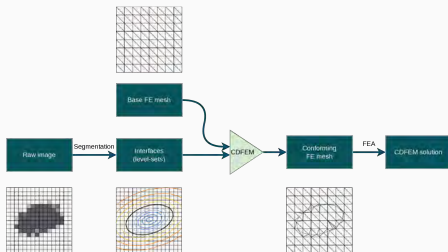


# Continuous decomposition FEM (cdfem)



- + Good geometric accuracy, reusing finite element ecosystem
- Elements quality, ill-conditioning, hard to extend to high order

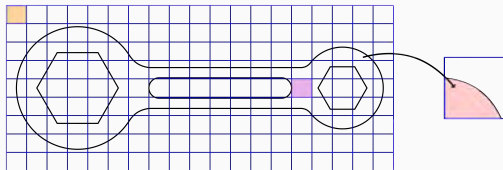
# Continuous decomposition FEM (cdfem)



- + Good geometric accuracy, reusing finite element ecosystem
- Elements quality, ill-conditioning, hard to extend to high order
- Tried TMOP fitting for high-order cdfem (which is a challenge), but faced a tendency to produce "badly placed" high-order nodes who killed the convergence

# Fictitious domain method

[Saulev 1963] [Parvizian et al. 2007] [Belytschko et al. 2003] [Burman et al. 2014]



- Geometry immersed in a background mesh (unfitted)
- Approximation defined over the background mesh (no decomposition)
- Outside elements excluded, inside elements standard quadrature
- Geometry taken into account through a dedicated quadrature rule on cut elements

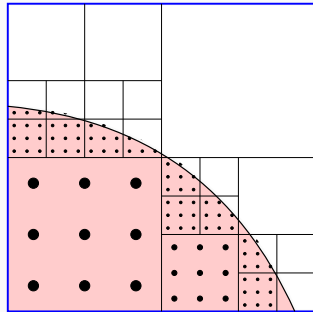
# Fictitious quadrature rule 1: sub-grid / space-tree

[Parvizian et al. 2007] [Dréau et al. 2010]

- Recursively subdivide the cut element ( $\ell = \text{leaf levels}$ )
- Then, either:
  - Quadrature rule on each leaf; discard points outside  $\Omega$
  - Local linear tessellation of  $\psi$  on each leaf; keep the part inside  $\Omega$

$\Rightarrow \ell \uparrow$ : better geometry, **but** point count grows fast

- + Simple, robust, works for any (even noisy) level-set
- Costly at high accuracy (compressible afterwards)



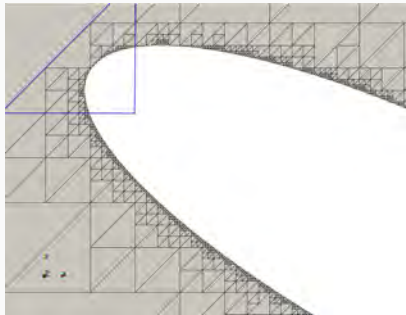
# Fictitious quadrature rule 1: sub-grid / space-tree

[Parvizia et al. 2007] [Dréau et al. 2010]

- Recursively subdivide the cut element ( $\ell$  = leaf levels)
- Then, either:
  - Quadrature rule on each leaf; discard points outside  $\Omega$
  - Local linear tessellation of  $\psi$  on each leaf; keep the part inside  $\Omega$

$\Rightarrow \ell \uparrow$ : better geometry, **but** point count grows fast

- + Simple, robust, works for any (even noisy) level-set
- Costly at high accuracy (compressible afterwards)

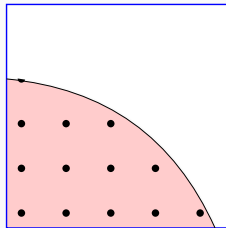


# Fictitious quadrature rule 2: moment fitting

[Sukumar & Mousavi 2010] [G.L. 2021]

- Choose  $n_q$  points  $x_i \in K$  arbitrarily; solve for weights  $w_i$  enforcing exactness on a polynomial basis  $\{\phi_k\}_{k=1}^{n_b}$  of order  $q$ :

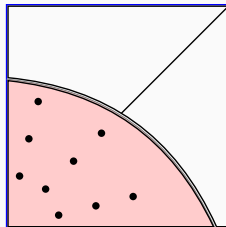
$$\sum_{i=1}^{n_q} w_i \phi_k(x_i) = \int_{K \cap \Omega} \phi_k(x) d\Omega, \quad k = 1, \dots, n_b$$



- Linear system for the weights  $w_i$  (points fixed, e.g. on a background grid)
- **Nothing guarantees**  $w_i \geq 0 \Rightarrow$  possible instabilities in nonlinear / dynamic problems
- + **Non-negative moment fitting**: enforce  $w_i \geq 0$  as an extra constraint (higher cost, but robust) — reusable across load steps / adaptivity

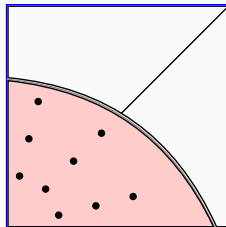
## Fictitious quadrature rule 3: higher order

- Use a local high-order representation of the interface (e.g. high-order level-set, NURBS, etc.)
- NURBS needs local fitting of the interface (e.g. least-squares) or robust CAD/mesh intersections



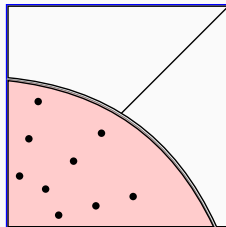
## Fictitious quadrature rule 3: higher order

- Use a local high-order representation of the interface (e.g. high-order level-set, NURBS, etc.)
- NURBS needs local fitting of the interface (e.g. least-squares) or robust CAD/mesh intersections
- Also tried local TMOP-based fitting for the quadrature cells (weaker geometrical requirements than for approximation), but did not work well (same reasons as for high-order cdfem)



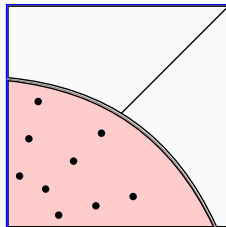
## Fictitious quadrature rule 3: higher order

- Use a local high-order representation of the interface (e.g. high-order level-set, NURBS, etc.)
- NURBS needs local fitting of the interface (e.g. least-squares) or robust CAD/mesh intersections
- Also tried local TMOP-based fitting for the quadrature cells (weaker geometrical requirements than for approximation), but did not work well (same reasons as for high-order cdfem)
- Finally settled on `algoim` (implicit height-function) quadrature, which is robust and efficient for **smooth level-sets**



## Fictitious quadrature rule 3: higher order

- Use a local high-order representation of the interface (e.g. high-order level-set, NURBS, etc.)
- NURBS needs local fitting of the interface (e.g. least-squares) or robust CAD/mesh intersections
- Also tried local TMOP-based fitting for the quadrature cells (weaker geometrical requirements than for approximation), but did not work well (same reasons as for high-order cdfem)
- Finally settled on `algoim` (implicit height-function) quadrature, which is robust and efficient for **smooth level-sets**
- But we did not use `mfem`'s implementation



## Fictitious quadrature rule 3: algoim (height-function)

[Saye 2015] [Saye 2022]

- Locally rewrite the interface as the graph of an **implicit height function** using the (high-order) level-set  $\psi$
- Reduces the cut integral to a nested sequence of 1D quadratures  $\Rightarrow$  **positive weights, known convergence order**
- Very efficient: far fewer points than sub-grid for the same accuracy

### Catch

Requires a **smooth, high-order** interpolation of  $\psi$ . An order- $q$  algoim rule does *not* exactly integrate a degree- $p = q$  polynomial (curvature effect): in practice  $q = 3$   $\mathcal{O}$  is needed,  $\mathcal{O}$  = FE polynomial order.

High-order smooth level-sets are hard to obtain for noisy / image-based (non-smooth) geometries  $\Rightarrow$  oscillations near corners.

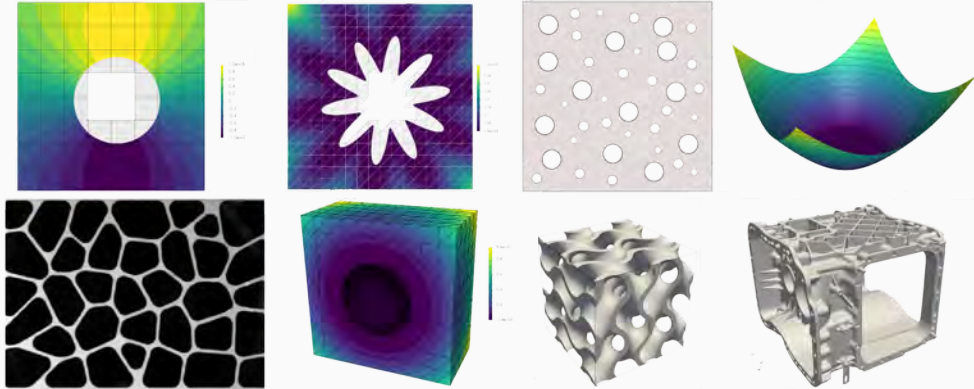
## Fictitious quadrature rules: summary

|                          | Sub-grid     | (NN) moment fitting | Algoim     |
|--------------------------|--------------|---------------------|------------|
| Positive weights         | yes          | only if “NN”        | yes        |
| Cost per accuracy        | high         | moderate–high       | <b>low</b> |
| Needs smooth $\psi$      | no           | no                  | <b>yes</b> |
| Reusable / adaptive      | compressible | yes (amortized)     | moderate   |
| Robust to noisy geometry | <b>yes</b>   | yes                 | no         |

- All three implemented in `mfem`, together with voxel and CDFEM
- Choice of quadrature rule is often **as important as** the choice of method itself — illustrated next

# Benchmarks

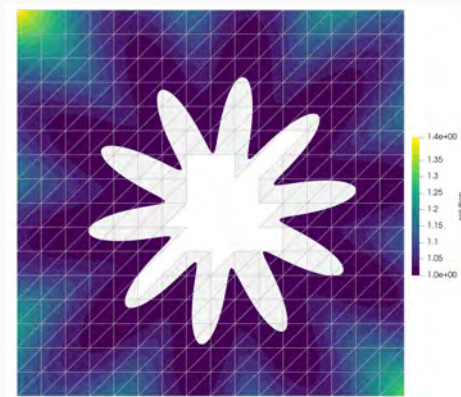
[Mawassy et al., Finite Elements in Analysis and Design 2026]



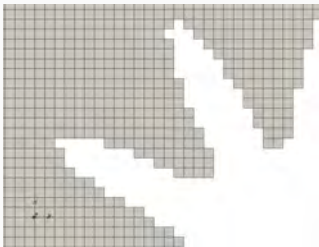
## 2D: flower-shaped free surface

$$\psi(r, \theta) = r_0 + \alpha \sin(w\theta) - r$$

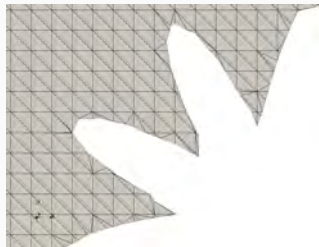
- High curvature variations
- Manufactured Laplace solution with **zero flux** on the interface
- Removes the bias of prescribing the exact flux



## 2D: flower-shaped free surface



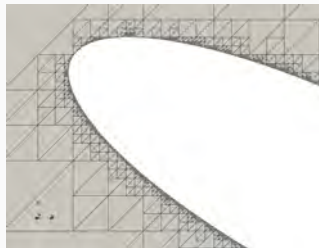
voxel ( $80 \times 80$ )



cdfem ( $32 \times 32$ )

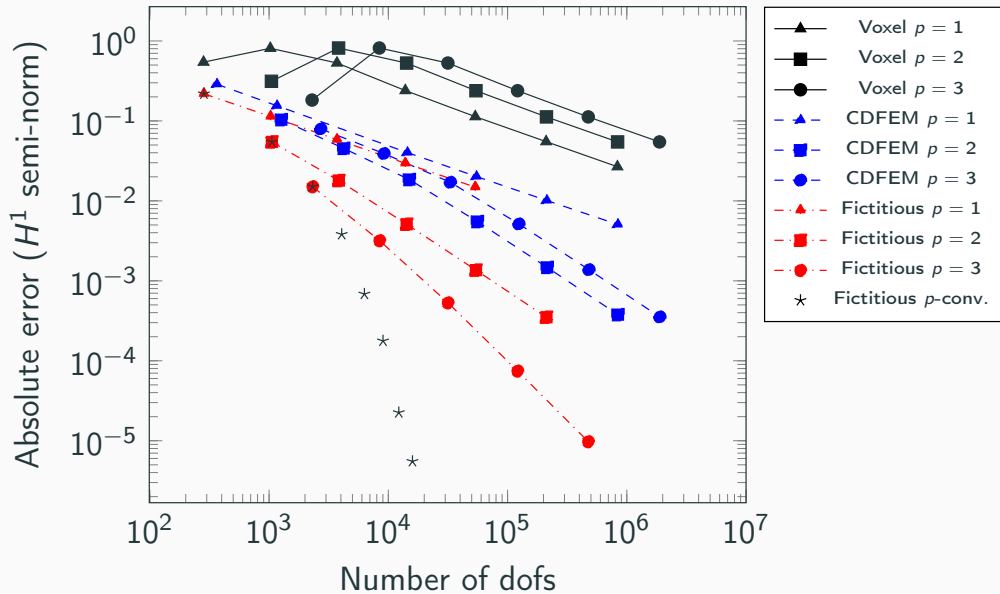


fictitious ( $16 \times 16$ )

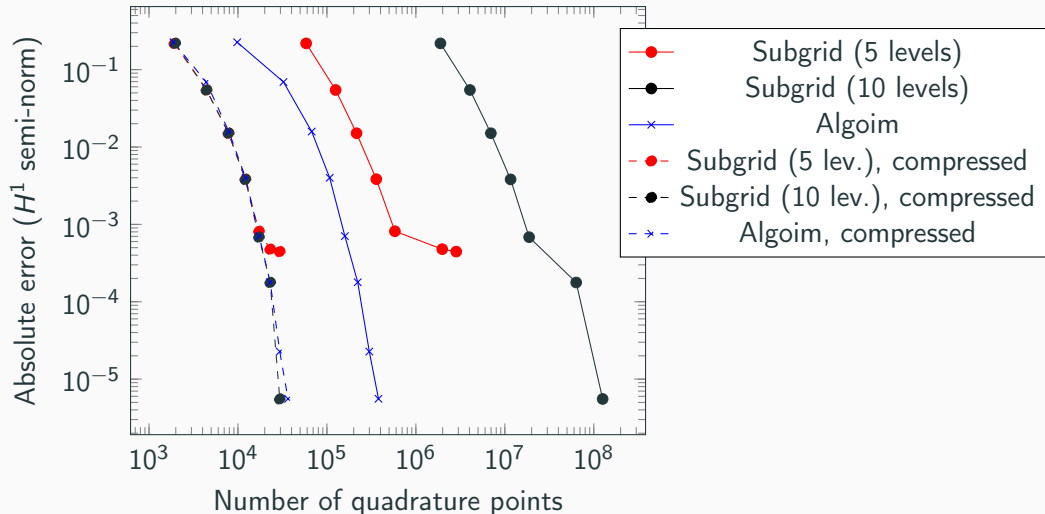


quad. cells (5 levels)

# Flower: $h$ - and $p$ -convergence

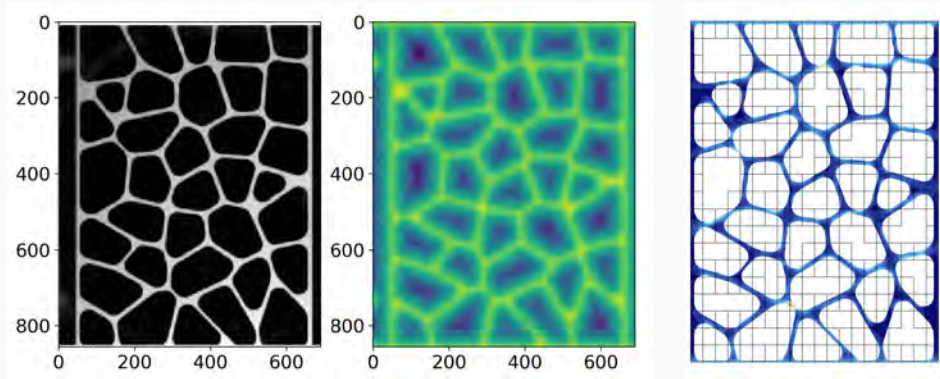


## Flower, p-convergence: cost of the quadrature rule



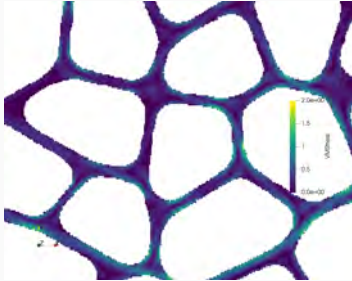
# Cellular material

[Rouwan et al. 2022]

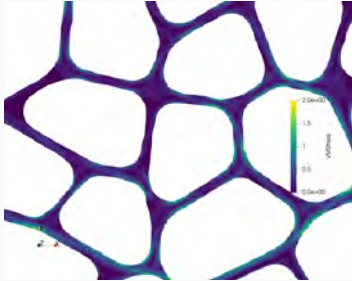


- Homogeneization of the elastic properties
- Kinematic uniform boundary conditions on the unit cell

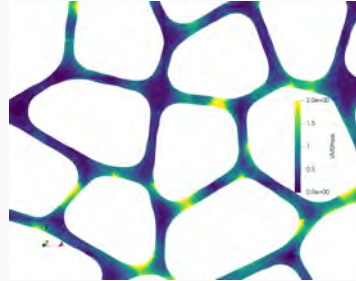
## Cellular material: raw stress field



voxel

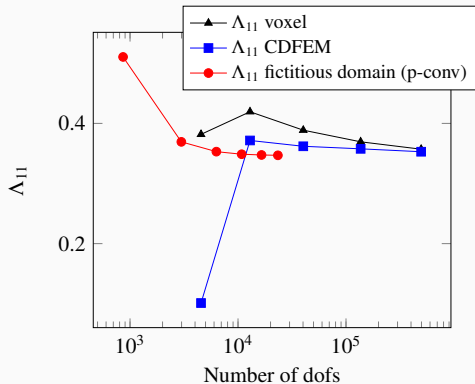


cdfem

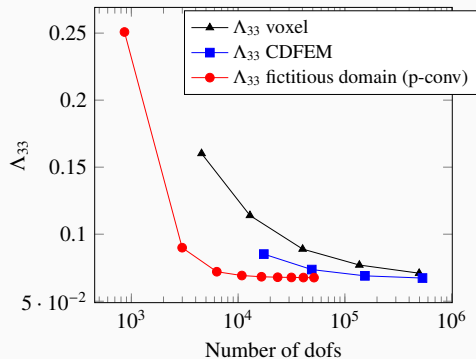


fictitious domain ( $p=9$ )

# Cellular material: convergence of the homogenized properties

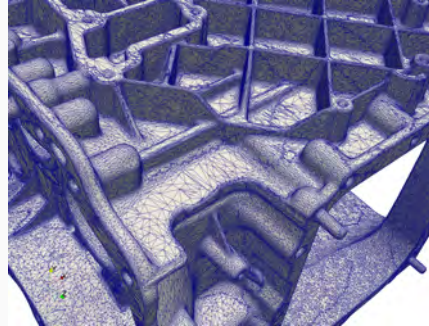
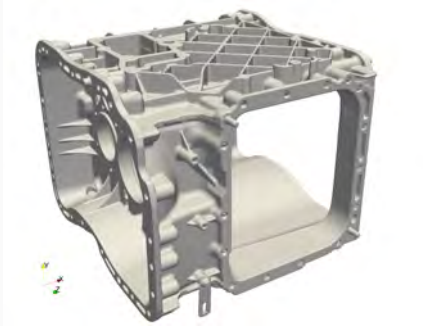


macro-traction



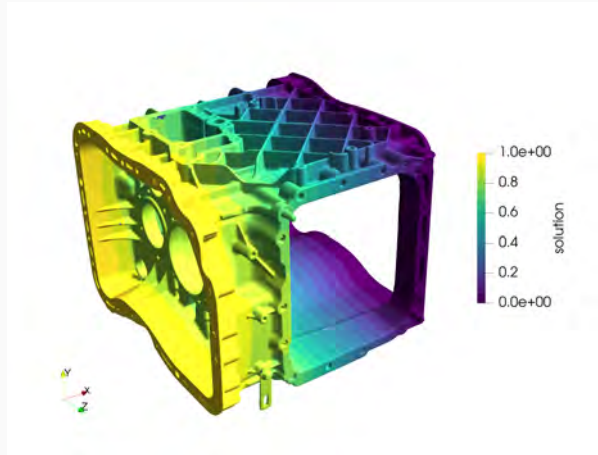
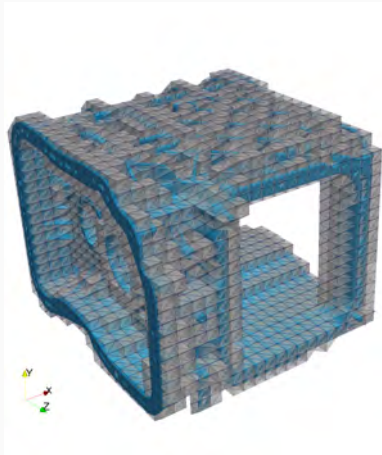
macro-shear

## Industrial 3D example: reverse-engineered gearbox

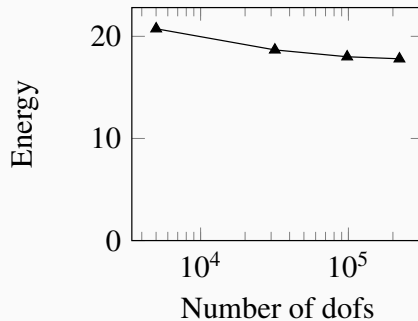
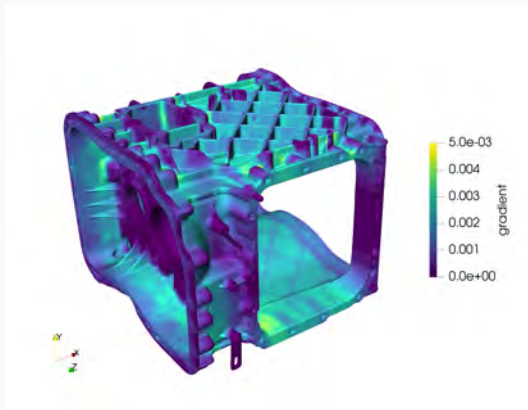


- Scanned mesh: 1.1M facets  $\rightarrow$  level-set on a  $300^3$  grid in  $< 1$  min (2019 computer)
- Thermal problem, 48 000 fourth-order tetrahedra
- Solution essentially converged already at  $p = 4$

# Industrial 3D example: reverse-engineered gearbox



# Industrial 3D example: reverse-engineered gearbox



- Three methods with minimal-pretreatment compared *in the same code* (mfem), on a wide range of 2D/3D scalar & vector problems
- Voxel-based FEM: easiest, but always outperformed for "small" problems
- CDFEM: best trade-off for low-order approximations
- Fictitious domain: best for high-order approximations and complex geometries
- Quadrature strategy strongly impacts fictitious domain performance:
  - Sub-grid: robust but expensive (compressible)
  - Algoim: efficient, but needs a smooth, high-order level-set

- Compare against recent alternatives:
  - CISAMR (Conforming-to-Interface Structured AMR)
  - X-Mesh method
- More efficient compression routines for sub-grid / moment-fitting rules
- hp level-set adaptivity for efficient algoim quadrature

Thank you for your attention!